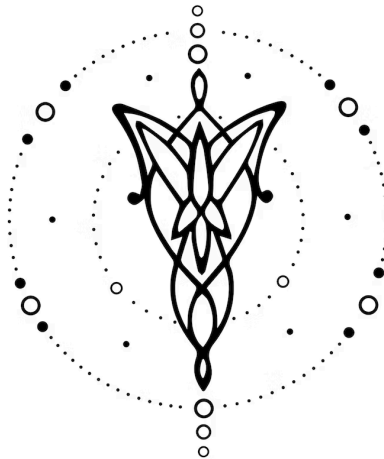




Especificación

Diseño e Implementación de un Lenguaje

14/06/2026

Apellidos	Nombres	Legajo	e-mail
Yelmini	Catalina	64133	cyelmini@itba.edu.ar
Gonzalez Porzio	Magalí	64028	mgonzalezporzio@itba.edu.ar
Ravera	Josefina	65690	jravera@itba.edu.ar
Grelle	Azul	65221	agrelle@itba.edu.ar

Tabla de contenidos

- Tabla de contenidos..... 2
- 1. Introducción..... 4
- 2. Modelo Computacional..... 4
 - 2.1. Dominio..... 4
 - 2.2. Lenguaje..... 4
 - Casos de prueba..... 7
- 3. Implementación..... 8
 - 3.1. Frontend..... 8
 - Analizador Léxico (Flex)..... 8
 - Analizador Sintáctico (Bison)..... 8
 - AST..... 9
 - 3.2. Backend..... 9
 - Análisis Semántico..... 9
 - Evaluación — Calculator..... 10
 - Generación de Código — Generator..... 10
 - 3.3. Adicionales..... 10
 - 3.4. Dificultades Encontradas..... 10
- 4. Futuras Extensiones..... 11
- 5. Conclusiones..... 11
- 6. Referencias..... 12
- 7. Bibliografía..... 12

1. Introducción

En el presente trabajo se desarrolló un compilador para un lenguaje de dominio específico (DSL) destinado a la creación y visualización de relojes analógicos. El lenguaje permite definir múltiples relojes con nombre, manipular su hora mediante operaciones aritméticas y de estilo, y generar una página HTML con relojes SVG animados que muestran la hora configurada y continúan girando en tiempo real.

El proyecto se llevó adelante en tres etapas. Primero se definieron la especificación y la gramática del lenguaje. Luego se implementó el frontend utilizando Flex y Bison junto con la construcción del AST en C. Finalmente, se desarrolló el backend encargado del análisis semántico, la evaluación del programa y la generación del código HTML+SVG.

A diferencia de la especificación inicial, la versión final no posee una instrucción `render` explícita. Todos los relojes declarados se renderizan automáticamente al finalizar la compilación, por lo cual dicha instrucción se vuelve innecesaria.

2. Modelo Computacional

2.1. Dominio

El dominio del lenguaje son los relojes analógicos como objetos con estado: hora, minuto y propiedades visuales, incluyendo colores de agujas, fondo, borde y tipo de numerales. El lenguaje permite operar sobre ese estado de forma imperativa: cada instrucción modifica el reloj activo, entendido como el último reloj declarado mediante `clock` [1].

El artefacto de entrada es un archivo fuente con extensión `.clock`. El artefacto de salida es un documento HTML autocontenido con CSS y SVG inline, de modo que puede abrirse directamente en un navegador sin depender de JavaScript ni de un runtime externo.

2.2. Lenguaje

El lenguaje se organiza como una secuencia de instrucciones. La instrucción `clock` declara un reloj con nombre y hora inicial, y desde ese momento ese reloj pasa a ser el reloj activo. Las instrucciones posteriores de tiempo, estilo, zona horaria o control de flujo actúan sobre ese reloj activo.

El conjunto final de construcciones incluye operaciones aritméticas, funciones temporales, estilos visuales, conversión de zona horaria, repetición e instrucciones condicionales. Las condiciones pueden usar comparadores simples sobre `hour` o `minute` y combinarse con `AND`, `OR` y `NOT`.

Instrucción	Sintaxis	Descripción
<code>clock</code>	<code>clock NOMBRE H:M</code>	Declara un reloj con nombre y hora inicial. Pasa a ser el reloj activo.
<code>add</code>	<code>add N hours minutes</code>	Suma N horas o minutos al reloj activo.
<code>sub</code>	<code>sub N hours minutes</code>	Resta N horas o minutos al reloj activo.
<code>set hour</code>	<code>set hour N</code>	Fija la hora del reloj activo en el rango 0-23.

set minute	set minute N	Fija los minutos del reloj activo en el rango 0-59.
round	round to 5 minutes	Redondea al múltiplo de cinco minutos más cercano.
next hour	next hour	Avanza al inicio de la próxima hora, fijando minutos en 0.
color	color COLOR	Define el color de las agujas.
background	background COLOR	Define el color de fondo de la esfera.
border	border COLOR	Define el color del borde del reloj.
numbers	numbers arabic roman	Define el tipo de numerales de la esfera.
Zona horaria	UTC±N -> UTC±M	Convierte la hora del reloj activo entre zonas horarias.
repeat	repeat N { instrucciones }	Repite un bloque de instrucciones N veces, con N mayor a 0.
if/else	if (cond) { ... } else { ... }	Ejecuta una rama condicional, else es opcional.

Los colores aceptados son `black`, `white`, `green`, `red` y `blue`. El estilo predeterminado utiliza agujas negras, fondo blanco, borde negro y numerales occidentales. La instrucción `numbers arabic` utiliza dígitos árabes orientales y `numbers roman` utiliza numerales romanos.

Los operadores de comparación disponibles son `==`, `!=`, `<`, `>`, `<=` y `>=`. La precedencia de operadores lógicos es `NOT > AND > OR`; por ejemplo, una condición de la forma `A OR B AND C` se interpreta como `A OR (B AND C)`.

Ejemplo 1 — Dos relojes con conversión de zona horaria

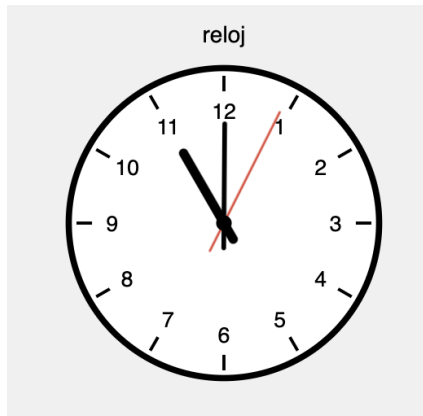
```
clock buenos_aires 10:30
UTC-3 -> UTC+0
color blue
background black
numbers roman

clock nueva_york 10:30
UTC-5 -> UTC+0
color red
background white
numbers arabic
```



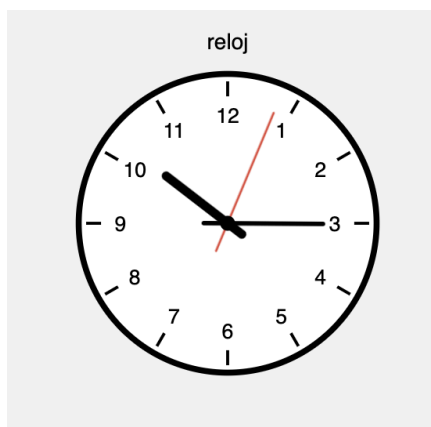
Ejemplo 2 — Control de flujo con if/else

```
clock reloj 10:45
if (minute >= 30) {
    next hour
} else {
    set hour 15
}
```



Ejemplo 3 — Repeat con aritmética

```
clock reloj 10:00
repeat 3 {
    add 5 minutes
}
```



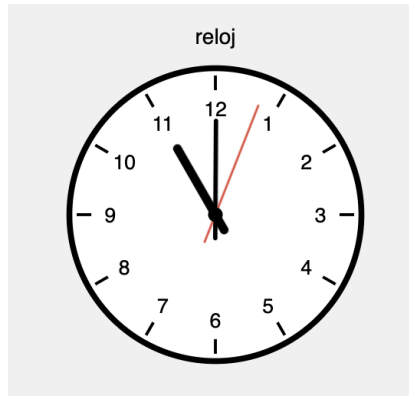
Ejemplo 4 — Operadores lógicos

```
clock reloj 10:45
```

```

if (minute >= 30 AND hour == 10) {
    next hour
}

```

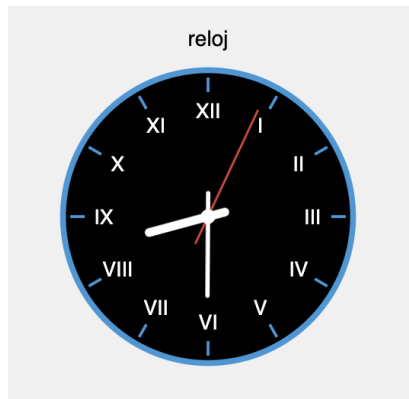


Ejemplo 5 — Todos los estilos

```

clock reloj 20:30
color white
background black
border blue
numbers roman

```



Casos de prueba

En total se desarrollaron 111 casos de prueba entre pruebas unitarias e integrales, cubriendo aspectos sintácticos, semánticos y funcionales del lenguaje.

Grupo	Cantidad	Cobertura
Stage II - unitarios	41	add/sub, set hour/minute, round, next hour, color, background, border, numbers, timezone, repeat, if/else y operadores lógicos.

Stage II - integración	8	Operaciones combinadas, estilos con <code>timezone</code> , <code>repeat</code> con múltiples instrucciones, ramas <code>true</code> y <code>false</code> de <code>if/else</code> , comentarios y múltiples relojes.
Grupo	Cantidad	Cobertura
Sintácticos - unitarios	36	Operador inválido, unidad inválida, cantidad negativa o no entera, faltantes de unidad/cantidad, <code>round</code> , <code>next</code> <code>hour</code> , <code>timezone</code> , <code>repeat</code> , <code>if</code> , color inválido, tipo de número inválido y operadores lógicos mal formados.
Sintácticos - integración	2	Casos de rechazo de integración del Stage II.
Semánticos - unitarios	20	<code>set hour</code> fuera de [0,23], <code>set minute</code> fuera de [0,59], <code>set hour/minute</code> no entero, instrucciones sin reloj activo (<code>add</code> , <code>sub</code> , <code>repeat</code> y funciones), <code>timezone</code> fuera de rango y <code>repeat</code> fuera de rango [1,MAX], hora/minuto inicial del clock fuera de rango, cantidad de <code>add/sub</code> fuera de rango (overflow) y condiciones de <code>if</code> con hora/minuto fuera de rango, incluso dentro de un <code>NOT</code> .
Semánticos - integración	4	Errores semánticos compuestos en programas más largos.

3. Implementación

La implementación se realiza en distintas etapas. Primero se analiza el archivo fuente con Flex y Bison para construir el AST. Luego se realiza la validación semántica, se calcula el estado final de los relojes y, por último, se genera el archivo HTML que representa el resultado.

```
.clock fuente → Flex (lexer) → Bison (parser) → AST → Análisis Semántico →
Calculator → Generator → output.html
```

3.1. Frontend

Analizador Léxico (Flex)

El analizador léxico se implementó en Flex. Su función es convertir el código fuente en una secuencia de tokens que luego utiliza Bison [2].

El lexer reconoce palabras reservadas del lenguaje, identificadores, literales enteros, operadores, delimitadores y comentarios. Los comentarios pueden ser de una línea o multilínea y son descartados durante esta etapa, por lo que no participan del análisis sintáctico posterior.

Analizador Sintáctico (Bison)

El parser se implementó con Bison y verifica que el programa cumpla las reglas definidas por la gramática [3].

La gramática soporta las distintas construcciones del DSL, incluyendo declaraciones de relojes, operaciones temporales, cambios de estilo, conversiones de zona horaria, repeticiones y estructuras condicionales.

Las expresiones booleanas utilizadas en las condiciones pueden combinar comparaciones simples mediante los operadores lógicos AND, OR y NOT. Para garantizar una interpretación consistente, se definió la precedencia lógica habitual, donde NOT tiene mayor precedencia que AND y AND mayor precedencia que OR.

La gramática soporta 14 tipos de instrucción mediante 17 producciones principales. Las operaciones add y sub tienen dos producciones cada una, separando hours y minutes. La instrucción if posee una producción con else y otra sin else.

Producción	Uso
clock NAME INTEGER:INTEGER	Declaración de reloj.
add INTEGER hours add INTEGER minutes	Suma de tiempo.
sub INTEGER hours sub INTEGER minutes	Resta de tiempo.
set hour INTEGER / set minute INTEGER	Asignación de componente temporal.
round to INTEGER minutes	Redondeo; solo 5 se acepta mediante RoundSemanticAction.
next hour	Avance a la próxima hora exacta.
color/background/border COLOR	Estilo visual.
numbers arabic roman	Tipo de numerales.
timezoneExpr -> timezoneExpr	Conversión de zona horaria.
repeat INTEGER { instructionList }	Repetición.
if (condition) { instructionList } else? { instructionList }	Condicional.

AST

El AST se modeló mediante una estructura `Instruction` que incluye un campo discriminante `InstructionType`. Cada nodo almacena el tipo de instrucción que representa y los campos necesarios para dicha instrucción.

3.2. Backend

Análisis Semántico

El análisis semántico verifica restricciones propias del dominio que no pueden expresarse únicamente mediante la gramática [4].

Entre las validaciones realizadas se encuentran la detección de relojes con nombres duplicados, el uso de instrucciones sin un reloj activo previamente declarado, la verificación de rangos válidos para horas y minutos, la validación de offsets de zona horaria y el control de parámetros inválidos en estructuras de repetición.

Para realizar estas verificaciones se emplean estructuras auxiliares que permiten registrar los relojes declarados y conservar correctamente el contexto de ejecución durante el recorrido del árbol sintáctico.

Evaluación — Calculator

`Calculator` recorre el AST y ejecuta cada instrucción, obteniendo el estado final de los relojes definidos en el programa. Durante esta etapa se ejecutan las operaciones temporales, los cambios de estilo, las conversiones de zona horaria y las estructuras de control.

Los relojes se almacenan en una estructura dinámica que permite incorporar nuevos elementos durante la ejecución. Las operaciones temporales se implementan mediante aritmética modular para manejar correctamente cambios de día, desbordamientos y valores negativos.

Una vez finalizada la evaluación, se obtiene una representación concreta del estado final de todos los relojes declarados, que será utilizada por la etapa de generación de código.

Generación de Código — Generator

El generador toma el estado final de los relojes y produce un archivo HTML autocontenido utilizando CSS y SVG para su representación visual [5]. La generación se implementa recorriendo la lista de estados finales de los relojes (`ClockStateList`) producida por el `Calculator` y emitiendo el HTML correspondiente para cada uno; el `Generator` [6] no accede al AST.

Cada reloj se renderiza de forma independiente utilizando elementos SVG para representar la esfera, los numerales, las agujas y los distintos estilos visuales [7]. Cuando existen múltiples relojes, estos se distribuyen automáticamente mediante un layout flexible.

La animación de las agujas se implementa mediante CSS `@keyframes` [8]. Para cada reloj, el generador emite reglas específicas para la aguja de hora, la aguja de minutos y la aguja de segundos. Cada animación rota desde el ángulo inicial calculado para el reloj hasta ese mismo ángulo más 360 grados.

De esta forma, el reloj se muestra en la posición correcta desde el momento en que se carga la página y luego continúa avanzando en tiempo real. La salida sigue siendo autocontenida: todo el HTML, el CSS y el SVG necesarios se generan en un único documento.

3.3. Adicionales

- Aguja de segundos siempre presente, roja y con período 60s.
- Posicionamiento automático de múltiples relojes mediante CSS Flexbox.
- Etiqueta de nombre debajo de cada SVG.
- Sanitización de nombres para IDs de DOM válidos.
- Tres tipos de numerales: occidentales por defecto, árabes orientales con `numbers arabic` y romanos con `numbers roman`.
- Modo Roman con agujas más cortas para no tapar los numerales.

3.4. Dificultades Encontradas

Durante la implementación surgieron distintos desafíos técnicos relacionados tanto con la representación gráfica como con la gestión interna del estado y la gramática.

1. **Transformaciones y centro de rotación en SVG/CSS:** una dificultad fue lograr que las agujas rotaran alrededor del centro correcto del reloj. Para resolverlo, los elementos SVG se construyeron alrededor del origen del `viewBox` y la rotación se aplicó mediante CSS, utilizando `transform-origin` y animaciones `@keyframes`.
2. **Punteros colgantes en Calculator:** Cuando `ClockStateList` crece con `realloc`, los punteros previos al arreglo quedan inválidos. Para evitar este problema, desde el diseño inicial se optó por rastrear el reloj activo por índice numérico en lugar de por puntero.
3. **Precedencia NOT > AND > OR en Bison:** En Bison la precedencia crece de arriba hacia abajo en las declaraciones `%left` y `%right`. Por eso `OR` debe declararse antes que `AND` y `NOT`.
4. **Animación sincronizada sin JavaScript:** para que cada reloj cargue mostrando la hora correcta, el generador calcula previamente los ángulos iniciales de cada aguja y los inserta en las reglas CSS generadas. Así, cada animación comienza desde la posición horaria correspondiente y continúa girando sin necesidad de JavaScript.
5. **Conflicto *dangling else*:** El único conflicto residual fue el *dangling else*. Bison lo resuelve prefiriendo `shift`, asociando el `else` al `if` más cercano.

4. Futuras Extensiones

- **Comportamiento por *tick*:** ejecutar snippets del DSL en cada *tick* del reloj para implementar DST y *leap seconds*.
- **Soporte de fechas** para complementar ajustes automáticos de DST.
- **Colores en hexadecimal libre (#RRGGBB)** en lugar de un enum de cinco colores.
- **Más unidades de tiempo**, como segundos y días.
- **Posicionamiento explícito** de relojes mediante grilla o posicionamiento relativo.
- **Exportación a formatos adicionales** como PNG y PDF.
- **Aguja de segundos configurable:** mostrar/ocultar y `color` personalizado.
- **Instrucción `numbers western`** para volver explícitamente al estilo predeterminado.

5. Conclusiones

Este proyecto nos permitió aplicar de forma práctica todas las etapas involucradas en la construcción de un lenguaje y su compilador (especificación léxica y gramatical, análisis sintáctico, construcción del AST, validación semántica, evaluación y generación de código). El principal desafío fue generar una salida que representara correctamente el comportamiento del lenguaje sin depender de componentes externos en tiempo de ejecución: animar las agujas con `@keyframes` exigió diseñar los elementos SVG alrededor del origen del `viewBox` y precalcular el ángulo inicial de cada aguja en las propias reglas CSS generadas, de modo que el reloj carga ya en la posición correcta y continúa girando sin JavaScript; y la gestión de memoria en el `Calculator` nos obligó a rastrear el reloj activo por índice para evitar punteros colgantes al crecer el arreglo con `realloc`.

A lo largo del proyecto pudimos comprender mejor cómo interactúan las distintas etapas del compilador, desde el análisis léxico hasta la generación de código. Si tuviéramos que empezar nuevamente el proyecto, optaríamos por una abstracción de tiempo más flexible y por permitir

colores hexadecimales libres. Sin embargo, el aprendizaje más importante fue comprobar cómo decisiones que parecen pequeñas al inicio terminan impactando en gran parte de la implementación posterior.

6. Referencias

- [1].  (2026-03-12) Modelo Computacional
- [2].  (2025-09-03, v1.0.0) Análisis Léxico .
- [3].  (2024-05-08, v0.1.0) Análisis Sintáctico
- [4].  (2024-09-25, v0.2.0) Análisis Semántico
- [5] MDN Web Docs. HTML: HyperText Markup Language.
<https://developer.mozilla.org/en-US/docs/Web/HTML>
- [6].  (2024-05-16, v0.1.0) Generación de Código
- [7] MDN Web Docs. SVG: Scalable Vector Graphics.
<https://developer.mozilla.org/en-US/docs/Web/SVG>
- [8] MDN Web Docs. @keyframes.
<https://developer.mozilla.org/en-US/docs/Web/CSS/@keyframes>

7. Bibliografía

-  (2026-03-07) Proyectos Anteriores .
-  (2026-03-08) Especificación
-  (2024-05-16, v0.1.0) Runtime
-  (2026-03-12) Proyecto Especial